

CH4 - Clip 2.

Storage와 Computing

01

Storage

02

Computing

1. Storage

Storage

개요

- Storage는 MLOps에서 데이터를 저장, 관리, 검색하는 중요한 구성 요소
- 데이터의 안정성, 접근성, 및 확장성을 보장하는 것이 핵심

중요성

- **데이터 보존:** 머신러닝 프로젝트에서 데이터는 가장 중요한 자산. 데이터의 안전한 저장은 데이터 손실을 방지하고, 언제든지 데이터에 대한 접근을 가능
- **접근성:** 효율적인 데이터 접근은 머신러닝 모델의 트레이닝과 테스트 시간을 단축. 빠른 데이터 접근은 프로젝트의 전반적인 생산성을 향상
- **확장성:** MLOps 프로젝트는 시간이 지남에 따라 데이터 양이 증가. 따라서, 스토리지 시스템은 확장 가능하고 유연해야함

Storage 유형

유형

- 클라우드 스토리지
- 분산 파일 시스템
- 데이터 웨어하우스
- 데이터 레이크

Storage 유형

클라우드 스토리지

- 예시 : Amazon S3, Google Cloud Storage, Azure Blob Storage
- 기능

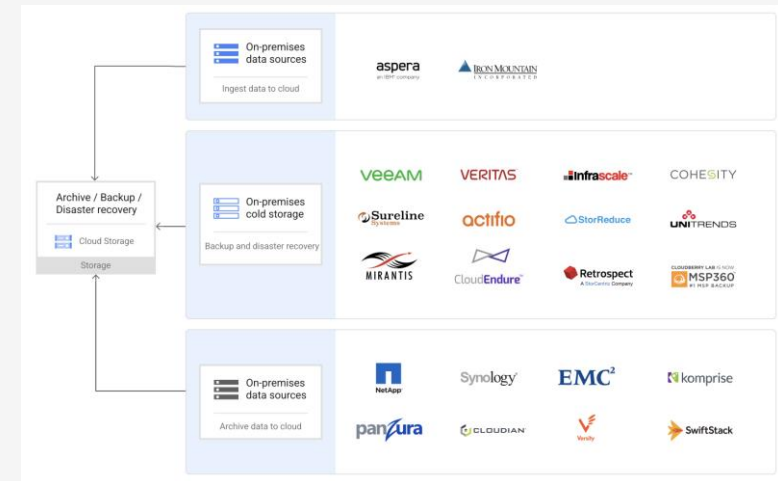
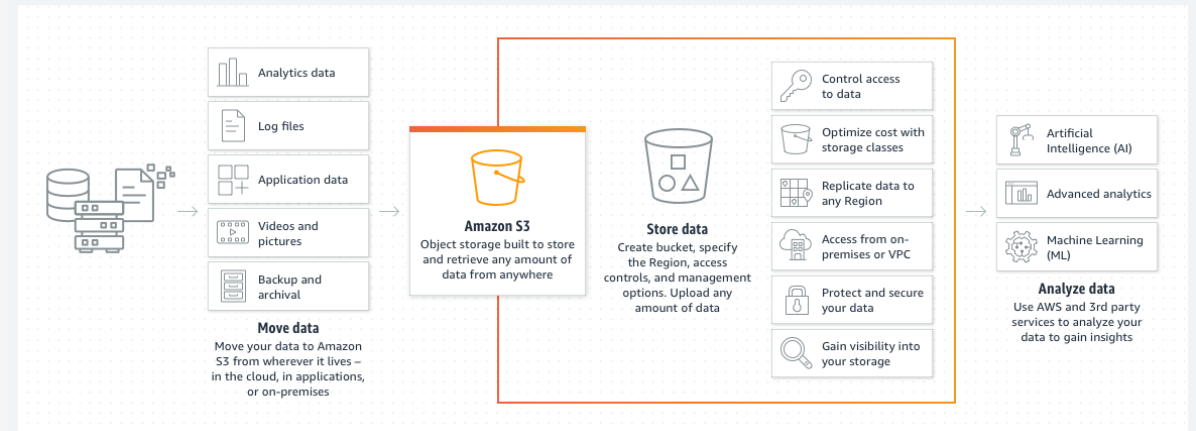
확장성: 거의 무한에 가까운 저장 공간 제공, 데이터 양에 따라 자유롭게 확장 가능.

내구성과 가용성: 데이터 복제 및 여러 지역에 걸친 분산 저장으로 높은 내구성 및 가용성 보장.

안정성: 고급 암호화 및 보안 프로토콜을 통해 데이터 보호.

비용 효율성: 사용한 만큼만 비용 지불, 다양한 저장 옵션 제공.

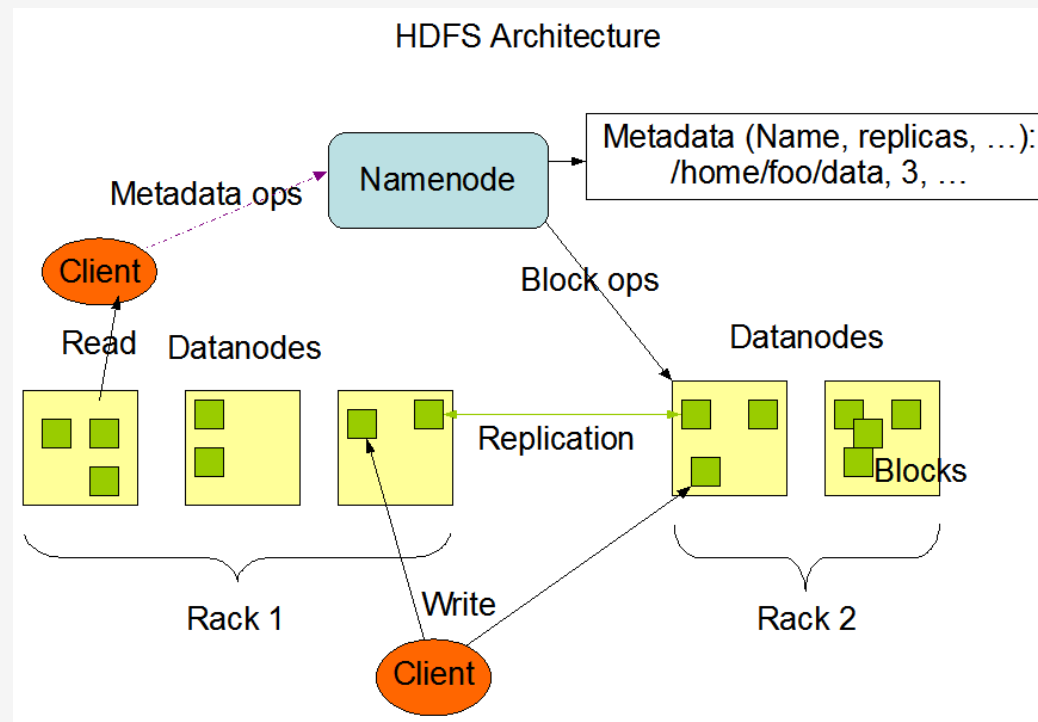
- **MLOps 활용 예:** 대규모 이미지 또는 비디오 데이터셋의 저장 및 분석, 글로벌 머신러닝 파이프라인에서의 데이터 공유.



Storage 유형

분산 파일 시스템

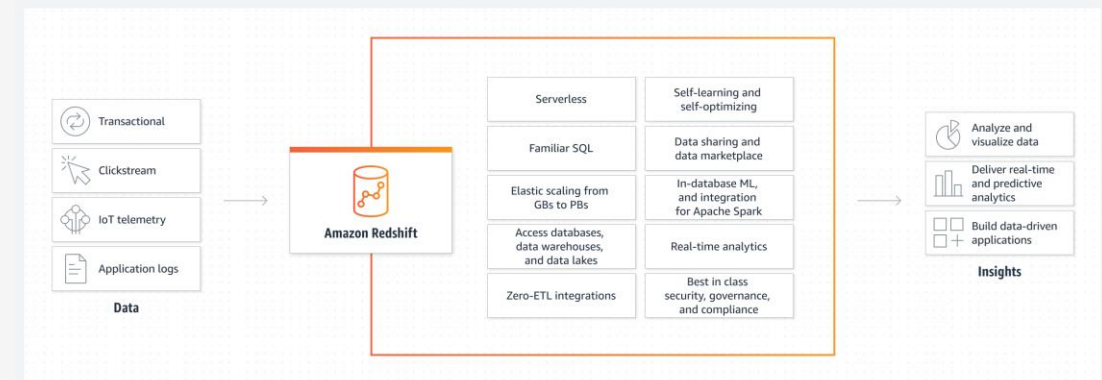
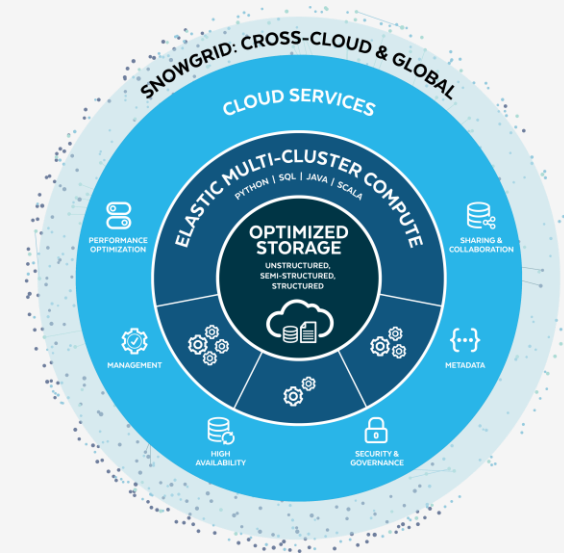
- 예시 : Hadoop Distributed File System (HDFS), GlusterFS
- 기능
 - 대규모 데이터 처리: 여러 노드에 걸쳐 대규모 데이터를 분산 저장하고 처리.
 - 고가용성: 데이터를 여러 노드에 복제하여 하나의 노드에 문제가 발생해도 데이터 손실 방지
 - 확장 가능한 아키텍처: 데이터 양이 증가함에 따라 쉽게 노드를 추가하여 시스템 확장 가능.
 - 비용 효율적: 오픈 소스 솔루션을 사용하여 비용 절감 가능.
- MLOps 활용 예: 대규모 빅 데이터 분석, 복잡한 데이터 처리 작업, 대용량 데이터의 실시간 처리 및 저장



Storage 유형

데이터 웨어하우스

- 예시 : Snowflake, Amazon Redshift, Google BigQuery
- 기능
 - 고속 쿼리 실행: 최적화된 쿼리 엔진을 통한 빠른 데이터 처리 및 분석.
 - 대규모 데이터 분석: 구조화된 대용량 데이터에 대한 효율적인 분석 및 저장
 - 사용 편의성: SQL 기반 쿼리로 사용자 친화적.
 - 보안과 규제 준수: 데이터 보안 및 다양한 규제 준수 옵션 제공.
- **MLOps 활용 예** : 비즈니스 인텔리전스, 대규모 데이터셋에 대한 복잡한 분석 및 리포팅



Storage 유형

데이터 레이크

- 예시 : AWS Lake Formation, Azure Data Lake

- 기능

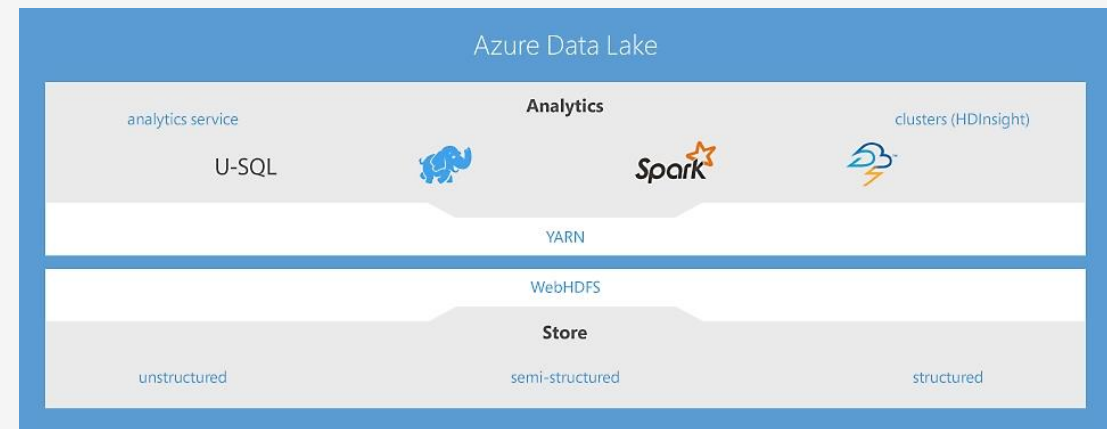
다양한 데이터 형식 지원: 구조화되지 않은 데이터에서부터 구조화된 데이터까지 다양한 형식의 데이터 저장 및 관리.

유연한 데이터 처리: 데이터 형식에 구애받지 않고 저장, 검색 및 분석 가능.

대규모 데이터 관리: 데이터 레이크는 방대한 양의 데이터를 효과적으로 관리할 수 있는 능력을 가짐.

비용 효율성: 사용한 만큼만 비용 지불, 다양한 저장 옵션 제공. 데이터 통합: 다양한 소스의 데이터를 하나의 위치에서 관리하고 분석.

- **MLOps 활용 예:** 대규모 이미지 또는 비디오 데이터셋의 저장 및 분석, 글로벌 머신러닝 파이프라인에서의 데이터 공유.



Storage와 MLOps

데이터 준비

- Model Training을 위한 데이터 준비 및 전처리

Model Registry

- 트레이닝된 Model 및 관련 파일 저장

Data Pipeline

- 지속적인 데이터 업데이트와 Model Re-Training 지원

Storage와 MLOps 사례

클라우드 기반 스토리지 활용

- 대기업에서는 AWS S3를 사용하여 전 세계적으로 분산된 데이터 센터에서 대량의 사용자 데이터를 저장 및 관리합니다.

데이터 레이크를 통한 유연한

데이터 관리 금융 기관에서는 데이터 레이크를 구축하여 다양한 형태의 거래 데이터를 효과적으로 관리하고 분석합니다.

2. Computing

Computing

개요

- Computing Resource는 데이터 처리, 모델 트레이닝 및 추론에 필요한 컴퓨팅 파워를 제공
- 높은 성능과 확장 가능한 리소스로 빠른 처리 속도 및 효율성을 보장하는 것이 중요

중요성

- **성능** : 높은 컴퓨팅 파워로 복잡한 계산 및 데이터 처리 가능
- **유연성** : 다양한 컴퓨팅 요구사항에 대응할 수 있는 리소스 제공
- **확장성** : 프로젝트 요구에 따라 컴퓨팅 리소스를 확장 또는 축소

Computing 유형

유형

- 클라우드 기반 컴퓨팅
- GPU/TPU
- 서버리스 컴퓨팅
- 컨테이너화된 컴퓨팅

Computing 유형

클라우드기반 컴퓨팅

- 예시 : AWS EC2, Google Compute Engine

- 기능

인스턴스 유형과 환경: 여러 가지 인스턴스 유형을 제공하여, 메모리, 컴퓨팅, 스토리지 최적화 등 다양한 필요에 맞게 선택 가능.

자동 스케일링: 트래픽이나 작업 부하에 따라 자동으로 리소스를 스케일링하여 효율성 및 비용 절감.

종합적인 보안: 네트워크 보안, 데이터 암호화, 접근 관리 등을 포함한 종합적인 보안 솔루션 제공.

전역 네트워크: 전 세계 데이터 센터를 통해 글로벌 액세스 및 가용성 보장.

- **MLOps 활용 예:** 대규모 딥러닝 모델을 위한 GPU 인스턴스 사용, 전 세계 사용자를 대상으로 하는 서비스의 글로벌 배포 및 로드 밸런싱



Amazon EC2



Google
Compute
Engine

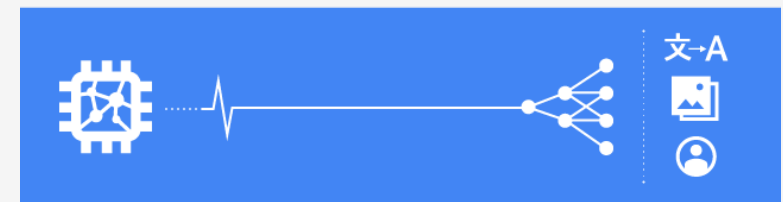
Computing 유형

GPU/TPU

- 예시 : NVIDIA의 Tesla 시리즈 GPU, Google Cloud TPU
- 기능
 - 딥러닝 최적화: 특히 딥러닝 작업을 위해 설계된 아키텍처로, 복잡한 신경망을 빠르게 트레이닝.
 - 고속 메모리 접근: 고속 메모리 및 대역폭으로 대량의 데이터를 신속하게 처리.
 - 에너지 효율적 설계: 고성능을 제공하면서도 에너지 소비를 최소화하는 효율적 설계.
 - 확장 가능한 인프라: 여러 GPU 또는 TPU를 연결하여 대규모 병렬 처리 구성 가능.
- **MLOps 활용 예:** 고해상도 이미지 인식, 복잡한 언어 모델 트레이닝, 대규모 데이터셋에서의 빠른 추론



Cloud TPU ^{BETA}



Google Cloud Platform

Computing 유형

서버리스 컴퓨팅

- 예시 : AWS Lambda, Google Cloud Functions, Azure Functions

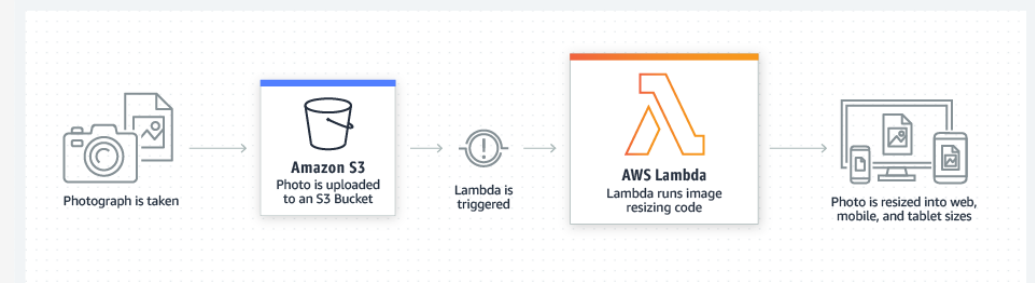
- 기능

코드 중심 실행: 서버 구성이나 관리 없이 바로 코드 실행에 집중 가능.

다양한 트리거 옵션: HTTP 요청, 데이터베이스 이벤트, 큐 메시지 등 다양한 이벤트에 의해 트리거되는 실행.

빠른 배포 및 업데이트: 새로운 코드 버전을 빠르게 배포하고 업데이트. 통합 에코시스템: 클라우드 서비스와의 긴밀한 통합으로 복잡한 작업을 쉽게 구성.

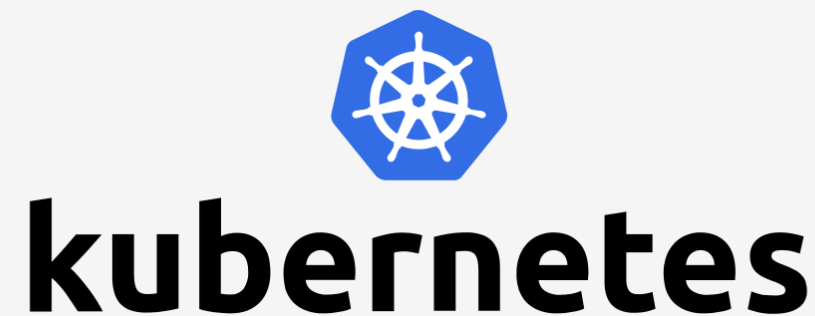
- **MLOps 활용 예:** 대규모 딥러닝 모델을 위한 GPU 인스턴스 사용, 전 세계 사용자를 대상으로 하는 서비스의 글로벌 배포 및 로드 밸런싱



Computing 유형

컨테이너화된 컴퓨팅

- 예시 : Docker 컨테이너, Kubernetes 클러스터
- 기능
 - 개발 및 배포 일관성: 모든 개발, 테스트, 프로덕션 환경에서 동일한 소프트웨어 환경 보장
 - 자동 오케스트레이션: Kubernetes와 같은 도구를 사용하여 컨테이너 배포, 관리, 스케일링 자동화
 - 마이크로서비스 아키텍처 지원: 각각의 컨테이너가 독립적인 서비스로 작동, 유연하고 확장 가능한 서비스 구축 지원
 - 리소스 효율성: 컨테이너는 가볍고 빠르며, 필요한 리소스만 사용하여 효율적인 운영 가능
- **MLOps 활용 예:** 다양한 머신러닝 모델을 컨테이너화하여 일관된 환경에서 트레이닝 및 추론, 모델의 쉬운 업데이트 및 롤백, 마이크로서비스 기반의 머신러닝 시스템 구축.



Computing과 MLOps

데이터 처리

- 대량의 데이터 전처리, 클렌징, 변환 등을 신속하게 수행

Model Training / Inference

- 복잡한 머신러닝 알고리즘에 적절한 계산 능력 제공, 예측 수행과 결과 제공을 빠르게 진행

Model Deployment / Serving

- 상품화 환경에서 모델의 안정적이고 효율적인 운영일 지원

Computing과 MLOps 사례

대규모 이미지 분석

- 의료 영상 분석 회사가 대규모의 MRI 및 CT 스캔 데이터를 처리

실시간 사용자 행동 예측

- 전자상거래 플랫폼에서 사용자 행동 데이터를 기반으로 구매 예측

자연어 처리 응용 프로그램

- 글로벌 기술 회사에서 다양한 언어의 텍스트 데이터 처리